Understanding the project

The control loop

1. SUMO advances vehicle movement and exposes traffic state through TraCI.

2. TrafficSignalEnv builds an 18-value observation from lane demand, current green, elapsed green, emergency presence and red ages.

3. The selected DQN or baseline proposes an approach and green duration.

4. The environment applies fairness/emergency safeguards and yellow/all-red clearance.

5. SUMO advances, the environment records performance, and the dashboard displays actual vehicle positions and signal states.

The safety wrapper is shared by the evaluated controllers. Therefore, a DQN proposal may be overridden. In simulation, state comes from SUMO ground truth, not camera detections.

Where to look

Folder or file	Purpose
dashboard/app.py	Local web server, simulation worker and state/control APIs
dashboard/templates, dashboard/static	HTML, styling and live city renderer
rl_env/traffic_env.py	Gymnasium environment, observations, actions, reward and signal safeguards
sumo_net	Intersection geometry, routes, scenario configuration and demand generator
train.py	DQN learning, reward normalization and checkpoints
eval/baselines.py	Fixed-cycle, density-adaptive and DQN policy interfaces
eval/select_model.py	Checkpoint selection on validation scenarios
eval/evaluate.py	Matched-scenario comparison of the three controllers
models/dqn_approach_best.zip	Selected four-approach checkpoint loaded by the dashboard
models/dqn_approach_best.json	Selection metadata and validation scores
eval/held_out_results_v3.json	Detailed measured results for the supplied model
notebooks/Traffic_Training_Colab.ipynb	Portable training, selection and evaluation workflow
vision	Separate pretrained YOLO vehicle detection demonstration
tests/test_simulation.py	Real-SUMO integration tests

How to interpret performance

Lower queue and waiting time are desirable; higher completed throughput is desirable. Always inspect unfinished/pending vehicles, collisions and teleports too. A controller that serves only easy trips can report low completed-trip wait while leaving a large backlog. Traffic control has no single classification-style accuracy percentage.

The current DQN uses a queue-based reward; do not describe it as a fully implemented PressLight controller. Density adaptive control is a rule-based baseline, not the trained neural model. The older two-axis checkpoints and README_legacy.md describe historical experiments, not the current four-approach controller. Progress_Report.docx is the original project document; use Project_Report.pdf for the current implementation summary.

What is in the delivery

Source code, intersection/route files, selected and historical model artifacts, sample vision assets, tests, training notebook, measured results and guides are included. Machine-specific venv, caches, logs and temporary build files are excluded. Dependencies install from requirements.txt; the ZIP is a portable source distribution, not a standalone executable.